

Model Quantization

LLM Fine-Tuning এর গাণিতিক ভিত্তি

সম্পূর্ণ বাংলায় — সহজ উদাহরণ থেকে গভীর তত্ত্ব পর্যন্ত

Quantization · Calibration · PTQ · QAT · LoRA · QLoRA

FP32 / FP16 / INT8

Scale Factor

Zero Point

Symmetric

Asymmetric

Fine-Tuning

LLaMA 2



বিষয়সূচি

১. সহজ ভাষায় পরিচিতি — Quantization কী ও কেন?

২. বিস্তারিত ব্যাখ্যা — 32-bit, 8-bit, মেমরি ফরম্যাট

৩. গাণিতিক তত্ত্ব — Scale Factor, Zero Point, Calibration

৪. Symmetric vs Asymmetric Quantization

৫. PTQ vs QAT — দুটি পদ্ধতির তুলনা

৬. Visual Diagram — Flow Chart ও Architecture

৭. বাস্তব ব্যবহারের উদাহরণ — Real-world Use Cases

৮. সুবিধা ও অসুবিধা — Pros & Cons

৯. Common Mistakes — নতুনরা যেখানে ভুল করে

১০. Related Topics — আগে ও পরে কী শিখবে

১১. Memory Tricks ও Quick Reference Card

Quantization কী?

Model Quantization মানে হলো — মেমরিতে সংরক্ষিত ডেটাকে **উচ্চতর মেমরি ফরম্যাট** (যেমন FP32) থেকে **নিম্নতর মেমরি ফরম্যাটে** (যেমন INT8) রূপান্তর করা। LLM মডেলে বিলিয়ন বিলিয়ন weight বা parameter থাকে, যেগুলো সাধারণত 32-bit ফরম্যাটে সেভ থাকে। Quantization করে সেগুলোকে 8-bit বা 16-bit এ নিয়ে আসা হয়।

💡 সহজ সংজ্ঞা

বড় মেমরি ফরম্যাটের ডেটাকে গাণিতিক উপায়ে ছোট মেমরি ফরম্যাটে রূপান্তর করার পুরো প্রক্রিয়াটিই হলো **Quantization**।

🔍 রান্নার উদাহরণ (বাস্তব জীবনের সাথে মিল)

কল্পনা করো তোমার কাছে একটি বিশাল **রান্নার রেসিপি বই** আছে — এতে প্রতিটি উপাদানের পরিমাণ মিলিগ্রাম পর্যন্ত নিখুঁতভাবে লেখা (এটা FP32 মডেল)। কিন্তু তুমি এটাকে একটি ছোট **পকেট কার্ডে** লিখতে চাও (INT8)। তখন তুমি পরিমাণগুলো গ্রামে রাউন্ড করে লিখবে।

⚠️ একটু নির্ভুলতা কমে, কিন্তু...

কার্ড ছোট হয় → সহজে পকেটে রাখা যায় → দ্রুত দেখা যায় → যেকোনো জায়গায় রান্না করা যায়। ঠিক তেমনি — Quantized মডেল একটু কম নির্ভুল, কিন্তু ছোট, দ্রুত, এবং যেকোনো ডিভাইসে চলে।

🔴 কোন সমস্যা সমাধান করে?

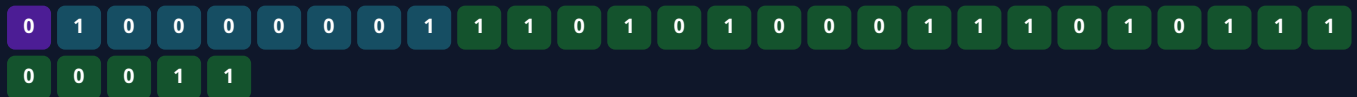
সীমিত RAM ও processing power

সমস্যা	কারণ	Quantization এর সমাধান
LLaMA 2 (70B) সাধারণ PC তে লোড হয় না	FP32 তে ~280GB RAM লাগে	INT8 তে ~70GB, INT4 তে ~35GB
Cloud server অনেক ব্যয়বহুল	বড় মডেল = বড় GPU দরকার	Local machine এ চালানো সম্ভব
Mobile/Edge device এ চলে না	ছোট quantized মডেল চলে	
Inference (response) অনেক ধীর	32-bit calculation ভারী	8-bit calculation অনেক দ্রুত
Fine-tuning কঠিন ও ব্যয়বহুল	বড় মডেল train করতে বিশাল GPU লাগে	Quantized মডেল fine-tune করা সহজ

1 2 3 4 32-bit (FP32) কী?

FP32 মানে **Single Precision Floating Point 32-bit**। মেমরিতে একটি সংখ্যা সেভ করতে ঠিক ৩২টি **binary digit** (0 অথবা 1) ব্যবহার হয়। Computer শুধু 0 ও 1 বোঝে — এই 32টি 0/1 মিলে যেকোনো floating point সংখ্যা (যেমন 7.32) প্রকাশ করা হয়।

FP32 এর ৩২টি Bit এর বিভাজন (উদাহরণ: 7.32)



■ Sign (1 bit) — ধনাত্মক/ঋণাত্মক
 ■ Exponent (8 bits) — সূচক/ঘাত
 ■ Mantissa (23 bits) — দশমিকের অংশ

অংশ	Bit সংখ্যা	কাজ	7.32 এর জন্য
Sign Bit	1 bit	ধনাত্মক (+) নাকি ঋণাত্মক (-) বোঝায়	0 = ধনাত্মক
Exponent	8 bits	সংখ্যার সূচক বা ঘাত সংরক্ষণ করে	7 এর অংশ
Mantissa	23 bits	দশমিকের পরের ভগ্নাংশ সংরক্ষণ করে	.32 এর অংশ
মোট	32 bits	একটি সম্পূর্ণ FP32 সংখ্যা	7.32

💡 8-bit (INT8) কেন 0 থেকে 255?

8-bit মানে একটি সংখ্যা রাখতে মাত্র ৪টি 0/1 ব্যবহার হয়। ৪টি সুইচ (ON/OFF) দিয়ে কতটি ভিন্ন কন্ফিগারেশন সম্ভব?

গাণিতিক হিসাব:

$2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 2^8 = 256$ টি কন্ফিগারেশন **Unsigned** (ধনাত্মক মাত্র): 0, 1, 2, 3 ... 254, 255 → মোট 256টি মান
Signed (ধনাত্মক + ঋণাত্মক): -128, -127 ... 0 ... 126, 127 → মোট 256টি মান

🔌 বিদ্যুৎ সুইচ উদাহরণ

৪টি সুইচ কল্পনা করো — প্রতিটি হয় OFF (0) নয়তো ON (1)।

1 সুইচ = 2 অবস্থা | 2 সুইচ = 4 অবস্থা | ৪ সুইচ = 256 অবস্থা (0000 0000 থেকে 1111 1111 পর্যন্ত)



মেমরি সাশ্রয়ের হিসাব — LLaMA 2 (70B Parameters)

Format	প্রতি Parameter	মোট মেমরি (70B)	ব্যবহারযোগ্যতা
FP32	4 bytes	~280 GB	সাধারণ PC তে অসম্ভব
FP16	2 bytes	~140 GB	খুব ব্যয়বহুল Server লাগে
INT8	1 byte	~70 GB	High-end GPU তে সম্ভব
INT4	0.5 bytes	~35 GB	Consumer GPU তে সম্ভব



গাণিতিক তত্ত্ব (Math / Theory)



Calibration কী?

Calibration হলো Quantization এর মূল প্রক্রিয়া — বড় রেঞ্জের মানগুলোকে ছোট রেঞ্জে "Squeeze" বা সংকুচিত করা। যেমন: 0-1000 রেঞ্জকে গাণিতিকভাবে 0-255 রেঞ্জে নিয়ে আসা।



Scale Factor সূত্র

সাধারণ সূত্র:

Scale Factor (S) = $(X_{\max} - X_{\min}) / (Q_{\max} - Q_{\min})$ যেখানে: $X_{\max}$, $X_{\min}$ →

Original floating point data এর সর্বোচ্চ ও সর্বনিম্ন মান $Q_{\max}$, $Q_{\min}$ → Target quantized range এর সর্বোচ্চ ও সর্বনিম্ন মান S → Scale Factor (এই দিয়ে ভাগ করে quantize হয়)

✓ Symmetric Quantization উদাহরণ (ধাপে ধাপে)

পরিস্থিতি: মডেলের weight রেঞ্জ 0 থেকে 1000। 8-bit Unsigned INT8 (0-255) তে convert করতে হবে।

ধাপ	হিসাব	ফলাফল
মূল রেঞ্জ	$X_{\min} = 0, X_{\max} = 1000$	ডেটা 0 থেকে 1000 এর মধ্যে
টার্গেট রেঞ্জ	$Q_{\min} = 0, Q_{\max} = 255$	8-bit unsigned এর range
Scale Factor	$(1000 - 0) / (255 - 0)$	S = 3.92
250 quantize করতে	$\text{round}(250 / 3.92) = \text{round}(63.7)$	→ 64
500 quantize করতে	$\text{round}(500 / 3.92) = \text{round}(127.5)$	→ 128
1000 quantize করতে	$\text{round}(1000 / 3.92) = \text{round}(255.1)$	→ 255
0 quantize করতে	$\text{round}(0 / 3.92) = \text{round}(0)$	→ 0



Dequantization (উল্টো করা)

Quantized value থেকে আবার Original value ফিরে পেতে:

Original $\approx$ Quantized $\times$ Scale Factor

উদাহরণ: $64 \times 3.92 = 250.88 \approx 250$ ✓

Zero Point কী এবং কেন দরকার?

Symmetric quantization এ ডেটা যদি শূন্য থেকে শুরু হয়, তাহলে শুধু Scale Factor যথেষ্ট। কিন্তু যদি ডেটায় **নেগেটিভ মান** থাকে (যেমন -20 থেকে 1000), তখন **Zero Point (Z)** নামের একটি অতিরিক্ত parameter দরকার।

Asymmetric Quantization সূত্র:

Scale Factor (S) = $(X_{\max} - X_{\min}) / (Q_{\max} - Q_{\min})$ Zero Point (Z) = $Q_{\min} - \text{round}(X_{\min} / S)$ Quantize করতে: $Q(x) = \text{round}(x / S) + Z$ Dequantize করতে: $x = S \times (Q - Z)$

✓ Asymmetric Quantization উদাহরণ (ধাপে ধাপে)

পরিস্থিতি: ডেটা রেঞ্জ -20 থেকে 1000। 8-bit Unsigned INT8 (0-255) তে convert করতে হবে।

ধাপ	হিসাব	ফলাফল
Scale Factor	$(1000 - (-20)) / (255 - 0) = 1020/255$	S = 4.0
-20 কে scale করলে	$\text{round}(-20 / 4.0) = \text{round}(-5)$	= -5 (range এ নেই!)
Zero Point নির্ণয়	$Z = 0 - (-5) = +5$	Zero Point = 5
-20 quantize করতে	$\text{round}(-20/4.0) + 5 = -5 + 5$	→ 0 ✓
0 quantize করতে	$\text{round}(0/4.0) + 5 = 0 + 5$	→ 5
500 quantize করতে	$\text{round}(500/4.0) + 5 = 125 + 5$	→ 130
1000 quantize করতে	$\text{round}(1000/4.0) + 5 = 250 + 5$	→ 255 ✓
Dequantize: 0 থেকে	$4.0 \times (0 - 5) = 4.0 \times (-5)$	= -20 ✓

🧠 Zero Point এর সহজ ব্যাখ্যা

Zero Point হলো সেই মান যা যোগ করলে নেগেটিভ সংখ্যা জোরপূর্বক target range এর শুরুতে (0 তে) চলে আসে। এটা মূলত পুরো number line টাকে ডানদিকে "shift" করার মতো।

✓ Symmetric Quantization

- ডেটা সুমমভাবে বণ্টিত (zero-centered)
- Zero Point সবসময় = 0
- শুধু Scale Factor দরকার
- হিসাব সহজ ও দ্রুত
- উদাহরণ: Batch Normalized weights

$$Q(x) = \text{round}(x / S) \quad x = Q \times S$$

⚡ Asymmetric Quantization

- ডেটা এক দিকে হেলানো (skewed)
- Zero Point $\neq 0$ (যেকোনো মান)
- Scale Factor + Zero Point দুটোই লাগে
- হিসাব একটু জটিল
- উদাহরণ: ReLU activation output

$$Q(x) = \text{round}(x / S) + Z \quad x = S \times (Q - Z)$$



বিস্তারিত তুলনা

বিষয়	Symmetric	Asymmetric
Zero Point	সবসময় 0	যেকোনো integer মান
নেগেটিভ ডেটা	সমানভাবে বণ্টিত হতে হবে	যেকোনো রেঞ্জে কাজ করে
Batch Normalization	হ্যাঁ, এখানে ব্যবহার হয়	না
ReLU activation	কম কার্যকর	বেশি কার্যকর
হিসাবের জটিলতা	সহজ	একটু জটিল
Memory overhead	কম (শুধু S)	একটু বেশি (S + Z)
নির্ভুলতা (asymmetric data)	কম নির্ভুল	বেশি নির্ভুল



ভিজুয়াল তুলনা

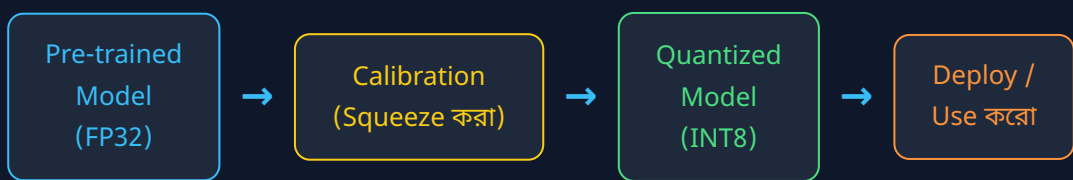
SYMMETRIC (শূন্য কেন্দ্রিক, সুমম বণ্টন):



৫ PTQ বনাম QAT

📦 Post-Training Quantization (PTQ)

ইতিমধ্যে **Train করা মডেলকে** Quantize করা। Weight গুলো Fixed — শুধু Calibration করে ছোট format এ convert করা হয়। নতুন করে train করতে হয় না।

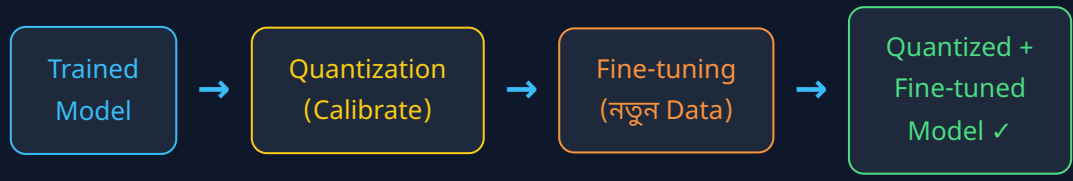


 **PTQ এর সমস্যা**

Calibration এর সময় কিছু information হারিয়ে যায় → Accuracy কিছুটা কমে। Fine-tuning এর জন্য PTQ একা যথেষ্ট নয়।

🎯 Quantization-Aware Training (QAT)

Quantize করার পর **নতুন ডেটা দিয়ে আবার Fine-tune** করা। Data loss পূরণ হয়, accuracy ভালো থাকে। Fine-tuning এ সবসময় QAT ব্যবহার হয়।



 **QAT এর সুবিধা**

নতুন training data দিয়ে fine-tune করার ফলে quantization এ যে accuracy কমে গিয়েছিল তা পুনরুদ্ধার হয়। LLaMA 2, Mistral ইত্যাদি মডেলের custom fine-tuning এ সবসময় QAT ব্যবহার হয়।

📊 PTQ vs QAT বিস্তারিত তুলনা

বিষয়	PTQ	QAT
পূর্ণ নাম	Post-Training Quantization	Quantization-Aware Training
কখন করা হয়	Training এর পরে	Quantize করার পরে
Training দরকার?	না	হ্যাঁ, fine-tuning লাগে
নতুন ডেটা লাগে?	না	হ্যাঁ, custom dataset লাগে

Accuracy	কিছুটা কমে	ভালো থাকে বা বাড়ে
গতি (process)	দ্রুত	ধীর (training লাগে)
GPU দরকার?	কম লাগে	Fine-tuning এর জন্য লাগে
Fine-tuning এ ব্যবহার	সাধারণত না	সবসময়
Use Case	Quick deployment	Custom data training

LoRA ও QLoRA এর সম্পর্ক

LoRA (Low-Rank Adaptation) → PTQ করা মডেলে নতুন ছোট weight layer যোগ করে fine-tune করে

QLoRA → Quantization (INT4) + LoRA = একসাথে, অনেক কম GPU তে fine-tuning সম্ভব

এই দুটি technique সম্পূর্ণ পরবর্তী নোটে বিস্তারিত আলোচনা হবে।



Visual Diagram — Flow Chart ও Architecture



Quantization Process Flow

QUANTIZATION PROCESS

FP32 মডেল (বিশাল)

Weight: 7.3200
Weight: -2.8910
Weight: 0.9832
(32 bits each)
Size: ~280 GB

Calibrate

INT8 মডেল (ছোট)

Weight: 64
Weight: 191
Weight: 127
(8 bits each)
Size: ~70 GB

পদক্ষেপ:

- ① X_max, X_min বের করো → ② Scale Factor হিসাব করো
- ③ Zero Point হিসাব করো → ④ প্রতিটি value: $\text{round}(x/S)+Z$



Bit Format তুলনা

Format	Bits	প্রতি Value	Range	মেমরি (70B)	
FP32	32	4 bytes	$\pm 3.4 \times 10^{38}$	~280 GB	✗
FP16	16	2 bytes	± 65504	~140 GB	⚠
BF16	16	2 bytes	$\pm 3.4 \times 10^{38}$	~140 GB	⚠
INT8	8	1 byte	-128 to +127	~70 GB	⚡
UINT8	8	1 byte	0 to 255	~70 GB	⚡
INT4	4	0.5 bytes	-8 to +7	~35 GB	✓



Symmetric vs Asymmetric Number Line

SYMMETRIC (Batch Normalized Data — শূন্য কেন্দ্রিক):

৭

বাস্তব ব্যবহারের উদাহরণ

Use Case	কোম্পানি / Project	কীভাবে ব্যবহার হয়	Format
Mobile এ LLM চালানো	Meta, Google	Llama 3, Gemma মোবাইলে offline ব্যবহার	INT4/ INT8
Laptop এ Chatbot	Ollama, LM Studio	GPU ছাড়া local LLM inference	GGUF INT4
Edge AI Camera	NVIDIA Jetson, Raspberry Pi	Real-time object detection	INT8
Smart Cooking AI	Custom (তোমার Project!)	ESP32 + Quantized LLM → ingredient control	INT8/ INT4
Medical Wearable	Various Healthcare	স্মার্টওয়াচে real-time health AI	INT4
Autonomous Vehicles	Tesla, Waymo	Edge chip এ real-time inference	INT8
E-commerce Chatbot	Custom (তোমার অন্য Project!)	FastAPI + Quantized LLM + MySQL RAG	INT8

✓ সুবিধা	✗ অসুবিধা
মেমরি 8x পর্যন্ত কমে (FP32→INT8)	কিছু Accuracy হারায় (PTQ তে)
Inference অনেক দ্রুত হয়	Calibration process জটিল
Mobile ও Edge device এ চলে	সব hardware সব format support করে না
Cloud server খরচ কমে	Asymmetric এ extra computation আছে
Fine-tuning সহজ ও সম্ভা হয়	QAT এ আবার train করতে হয় (সময় লাগে)
Model download size ছোট হয়	Integer overflow এর ঝুঁকি আছে
Energy consumption কমে	Quantization scheme manually বেছে নিতে হয়

✗ ভুল ১: "32-bit মানে 0 থেকে 1000 পর্যন্ত"

32-bit এর নিজস্ব কোনো নির্দিষ্ট "range" নেই। 1000 ছিল শুধু উদাহরণের জন্য একটি কাল্পনিক weight মান। FP32 এর আসল range হলো $\pm 3.4 \times 10^{38}$ — অসাধারণ বড়!

✓ **সঠিক ধারণা:** 8-bit এর range (0-255) হলো fixed ও গাণিতিক ($2^8=256$)। FP32 এর "0 থেকে 1000" ছিল উদাহরণের ডেটা, format এর range নয়।

✗ ভুল ২: Asymmetric এ শুধু Scale Factor ব্যবহার করা

নেগেটিভ ডেটার জন্য শুধু Scale Factor দিয়ে ভাগ করলে নেগেটিভ quantized value আসে — যা unsigned INT8 range (0-255) এ থাকে না।

✓ **সঠিক পদ্ধতি:** Asymmetric এ অবশ্যই Scale Factor + Zero Point দুটোই ব্যবহার করতে হবে।

✗ ভুল ৩: "Runtime এ Python compiler এর মতো auto convert হবে"

Python code ছোট (কিলোবাইট) — runtime এ binary করা সহজ। কিন্তু LLM এর weight বিলিয়ন বিলিয়ন সংখ্যা (~280GB FP32)। এত বড় ডেটা runtime এ memory তে load করেই convert করা সম্ভব নয়।

✓ **সঠিক পদ্ধতি:** Deployment এর আগেই model কে quantize করে রাখতে হয়। তারপর ছোট quantized model লোড করা হয়।

✗ ভুল ৪: Fine-tuning এ PTQ ব্যবহার করা

PTQ শুধু existing weight compress করে — নতুন data এর জন্য মানানসই নয়। Accuracy কমে যাবে।

✓ **সঠিক পদ্ধতি:** Custom dataset দিয়ে fine-tuning এ সবসময় QAT ব্যবহার করো।

✗ ভুল ৫: Quantization মানে model training করা

Quantization শুধু number format পরিবর্তন করে — model এর architecture বা behavior পরিবর্তন করে না। এটা training নয়, compression।

✓ **সঠিক ধারণা:** Quantization = compression। Fine-tuning = model শেখানো।

📖 আগে কী জানা দরকার?

- Neural Network ও Weight/Bias ধারণা
- Floating Point number system (FP32, FP16)
- Binary number system (0 ও 1)
- Matrix multiplication basics
- Batch Normalization
- Forward ও Backward Propagation
- Loss Function ও Gradient Descent

🚀 পরে কী শিখবে?

- **LoRA** — Low-Rank Adaptation fine-tuning
- **QLoRA** — Quantization + LoRA combined
- **PEFT** — Parameter Efficient Fine-Tuning
- **GGUF/GGML** — Quantized model file format
- **bitsandbytes** — Python quantization library
- **Hugging Face transformers** — quantization API
- **GPTQ, AWQ** — Advanced quantization methods

🔧 Tools ও Libraries

Tool	ব্যবহার	কোড উদাহরণ
bitsandbytes	4-bit ও 8-bit quantization	<code>load_in_8bit=True</code>
Hugging Face	BitsAndBytesConfig দিয়ে quantize	<code>BitsAndBytesConfig()</code>
GGUF (llama.cpp)	CPU তে quantized model চালানো	<code>q4_0, q8_0 format</code>
TensorFlow Lite	Mobile deployment	<code>TFLiteConverter</code>
Ollama	Local PC তে LLM চালানো	<code>ollama run llama3</code>

🧠 মনে রাখার সহজ কৌশল



Trick ১ — বোতল ও ট্যাঙ্কি

32-bit = বাড়ির ছাদের বিশাল পানির ট্যাঙ্কি (অনেক পানি ধরে)

8-bit = ছোট পানির বোতল (255 ml পর্যন্ত ধরে)

Quantization = ট্যাঙ্কির পানি Scale Factor (ফানেল) দিয়ে বোতলে ঢালা

Zero Point = বোতলের মুখ যদি মেঝে থেকে উঁচুতে থাকে, তাহলে নিচে থেকে তুলে দেওয়া



Trick ২ — সুইচের হিসাব

n-bit = n টি ON/OFF সুইচ

8 সুইচ = $2^8 = 256$ কম্বিনেশন = 0 থেকে 255 পর্যন্ত

32 সুইচ = $2^{32} = 4$ billion কম্বিনেশন = বিশাল range



Trick ৩ — Scale Factor মনে রাখা

Scale Factor = "বড় scale ÷ ছোট scale"

যেমন: মিটার → সেন্টিমিটার তে convert করতে ÷ 100 (এটাই scale factor)

1000 range কে 255 range এ আনতে ÷ 3.92 (scale factor)



Trick ৪ — PTQ vs QAT সহজে মনে রাখা

PTQ = "Post" = পরে করা = শুধু compress করা = accuracy কমে

QAT = "Aware Training" = সচেতনভাবে train করা = accuracy ঠিক থাকে

📌 ১ লাইনে সারসংক্ষেপ

"Quantization = বড় মডেলকে গাণিতিকভাবে ছোট করা —
Scale Factor ও Zero Point দিয়ে FP32 থেকে INT8 এ নিয়ে আসা —
যাতে সাধারণ ডিভাইসে দ্রুত ও সস্তায় AI চালানো যায়।"

⚡ Quick Reference Card

Scale Factor (S)	$(X_{\max} - X_{\min}) / (Q_{\max} - Q_{\min})$
Zero Point (Z)	$Q_{\min} - \text{round}(X_{\min} / S)$ [Asymmetric এ]
Quantize: $Q(x)$	$\text{round}(x / S) + Z$
Dequantize: x	$S \times (Q - Z)$
FP32 range	$\pm 3.4 \times 10^{38}$ 32 bits per value 4 bytes
INT8 unsigned	0 to 255 8 bits $2^8 = 256$ values
INT8 signed	-128 to 127 8 bits $2^8 = 256$ values
Memory savings	FP32 → INT8 = ৭৫% কম মেমরি (৪x ছোট)
Symmetric	Zero Point = 0 সুষ্ম ডেটার জন্য
Asymmetric	Zero Point $\neq$ 0 নেগেটিভ বা skewed ডেটার জন্য
PTQ	Post-Training Quantization No training Accuracy কমে
QAT	Quantization-Aware Training Fine-tune করে Accuracy ভালো
পরবর্তী বিষয়	LoRA → QLoRA → PEFT → GGUF